

Firestore & NoSQL

Développement d'applications mobiles

Daniel Schreurs

SQL : Connecting Data

Ce qu'on souhaite modéliser

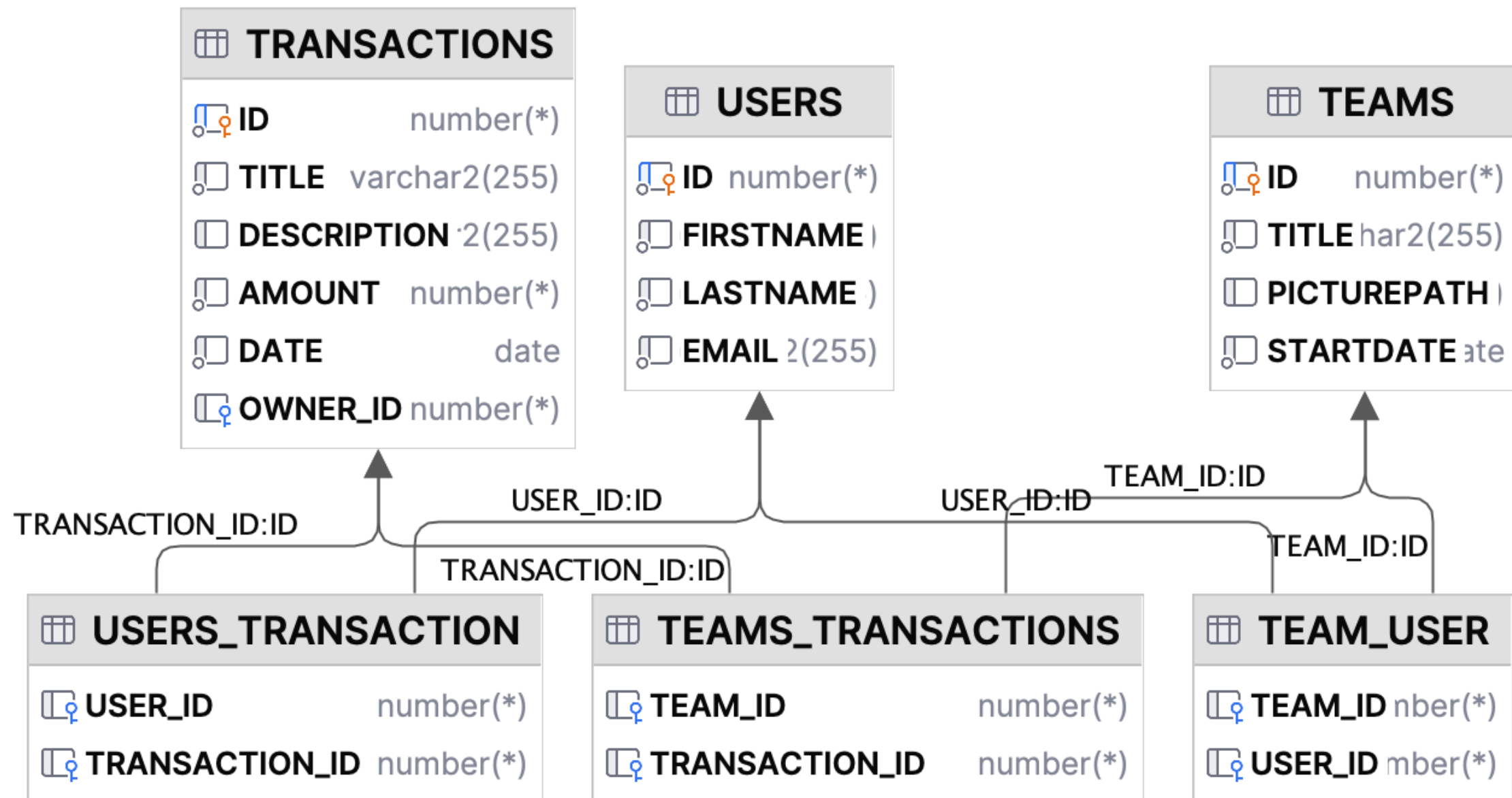
- Les utilisateurs sont membres de groupes
- Chaque groupe est constitué d'utilisateurs
- Au sein d'un groupe, ajouter des transactions :
- Des montants
- Qui concernent d'autres utilisateurs du même groupe



Les formes de normalisation

- **Première Forme Normale (1FN)**
 - Élimination des valeurs multiples.
 - Chaque attribut contient une seule valeur.
- **Deuxième Forme Normale (2FN)**
 - Respect de la 1FN.
 - Tous les attributs non-clés dépendent de la clé primaire.
- **Troisième Forme Normale (3FN)**
 - Respect de la 2FN.
 - Élimination des dépendances transitives.
- **Forme Normale de Boyce-Codd (BCNF)**
 - Respect de la 3FN.
 - Clés candidates déterminent tous les attributs non-clés.

Schéma de la base de données



Requête SQL

Join'ing creates Clarity!



```
select FIRSTNAME, LASTNAME  
from ...
```

	☐ FIRSTNAME	☐ LASTNAME	☐ TITLE
		Lovelace	 Code Magicians
1	Ada	Turing	 Code Magicians
2	Alan	Torvalds	 Web Weavers
3	Linus	Hopper	 Code Magicians
4	Grace	Hamilton	 Web Weavers
5	Margaret	Berners-Lee	 Web Weavers
6	Tim	Lovelace	 Game Gurus
7	Ada	Hopper	 Game Gurus
8	Grace	Hamilton	 Game Gurus
9	Margaret	Lovelace	✨ nouveau groupe
10	Ada	Hopper	✨ nouveau groupe
11	Grace	Hamilton	✨ nouveau groupe
12	Margaret		

```
U.USER_ID  
;
```

Gestion des valeurs dynamiques avec des vues

```
CREATE or replace VIEW FullTeams AS
SELECT T.*,
       (SELECT SUM(TRANSACTIONS.AMOUNT)
        FROM TEAMS_TRANSACTIONS
         inner join TRANSACTIONS ON TEAMS_TRANSACTIONS.TRANSACTION_ID = TRANSACTIONS.ID
        WHERE TEAMS_TRANSACTIONS.TEAM_ID = T.ID) AS Total
FROM TEAMS T;
```

ID	TITLE	PICTUREPATH	STARTDATE	TOTAL
1	Code Magicians	assets/img/team-1.png	2023-08-31	1350
2	Web Weavers	assets/img/team-2.png	2023-09-15	1700
4	nouveau groupe	assets/img/team-4.png	2023-09-30	1015
3	Game Gurus	assets/img/team-3.png	2023-09-30	1015

Vos groupes

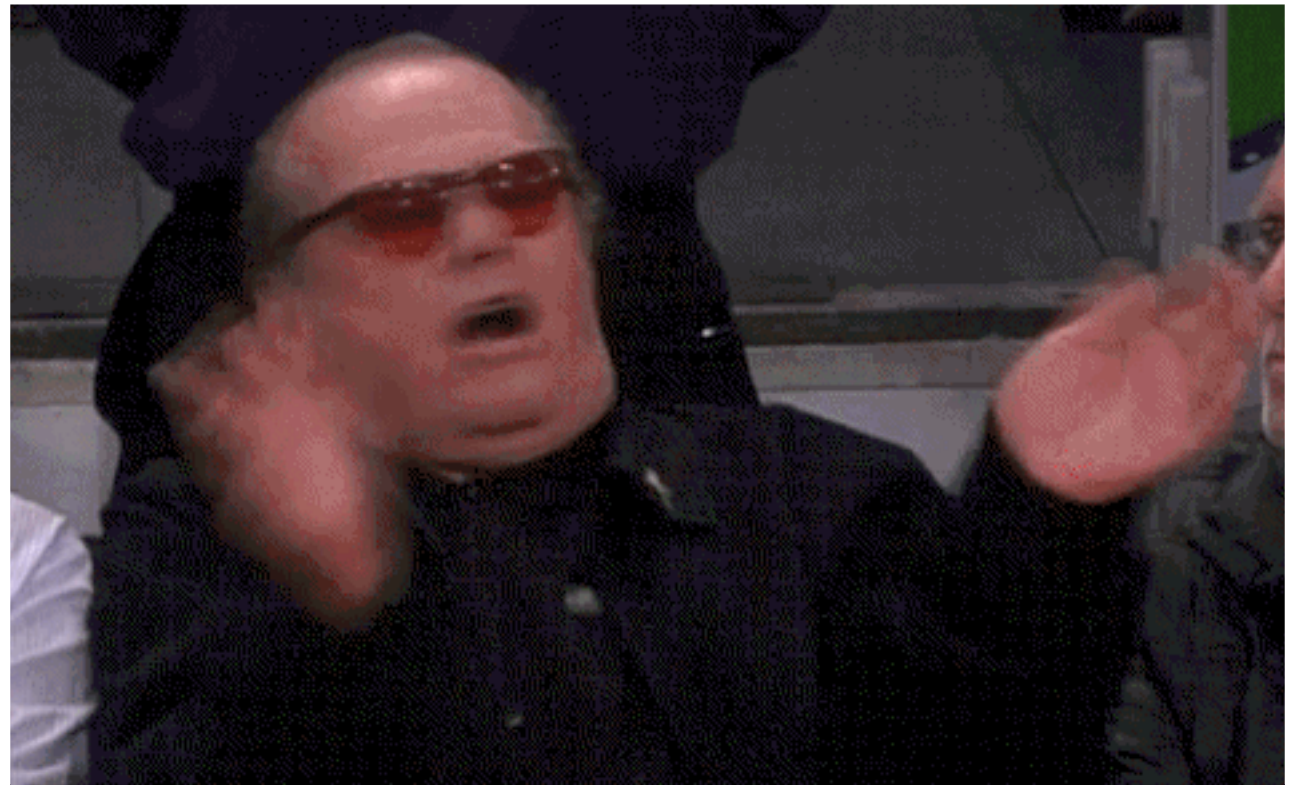
 **Game Gurus** 1.015€



14€
3 participants Depuis le 30/09/2023

^ Masquer les transactions

NoSQL 🧘



Points forts

- Flexibilité
- Évolutivité
 - Load balancing
 - Changer la structure en cours de route 🤯
- Données non structurées
 - Lectures très rapides
 - Requêtes simplistes !

Points faibles

Tout se paye...

- La mise à jour des données
 - Il faut mettre à jour toutes les données en double
 - Risque d'oublier des mises à jour... 🤯
- Redondance des données
- Ce qui paraît simple ne l'est pas au final

Concrètement

Tout va bien

+ Commencer une collection

teams >

users

+ Ajouter un document

1ZPAir3V3ZxdEyTMY0WE >

RHta1A1kGBRfNye1fRd7

aSnY02YqMrzAr51bfRHo

up9NwPKroIkNUIINKXb07

+ Commencer une collection

+ Ajouter un champ

2 "Networking"

title: "🌐 Web Weavers"

transactions

0

amount: 20

concerns

0

email: "alan@example.com"

firstName: "Alan"

lastName: "Turing"

toto: "titi"

1

email: "linus@example.com"

firstName: "Linus"

lastName: "Torvalds"

2

email: "tim@example.com"

firstName: "Tim"

lastName: "Berners-Lee"

date: 21 août 2023 à 00:00:00 UTC+2

description: "Achats préventifs pour la soirée"

owner

email: "tim@example.com"

firstName: "Tim"

lastName: "Berners-Lee"

title: "Pansements Ampoules"

1

amount: 50

(default)

users

alan@example.com

+ Commencer une collection

+ Ajouter un champ

email: "alan@example.com"

firstName: "Alan"

lastName: "Turing"

+ Commencer une collection

+ Ajouter un document

ada@example.com

alan@example.com >

grace@example.com

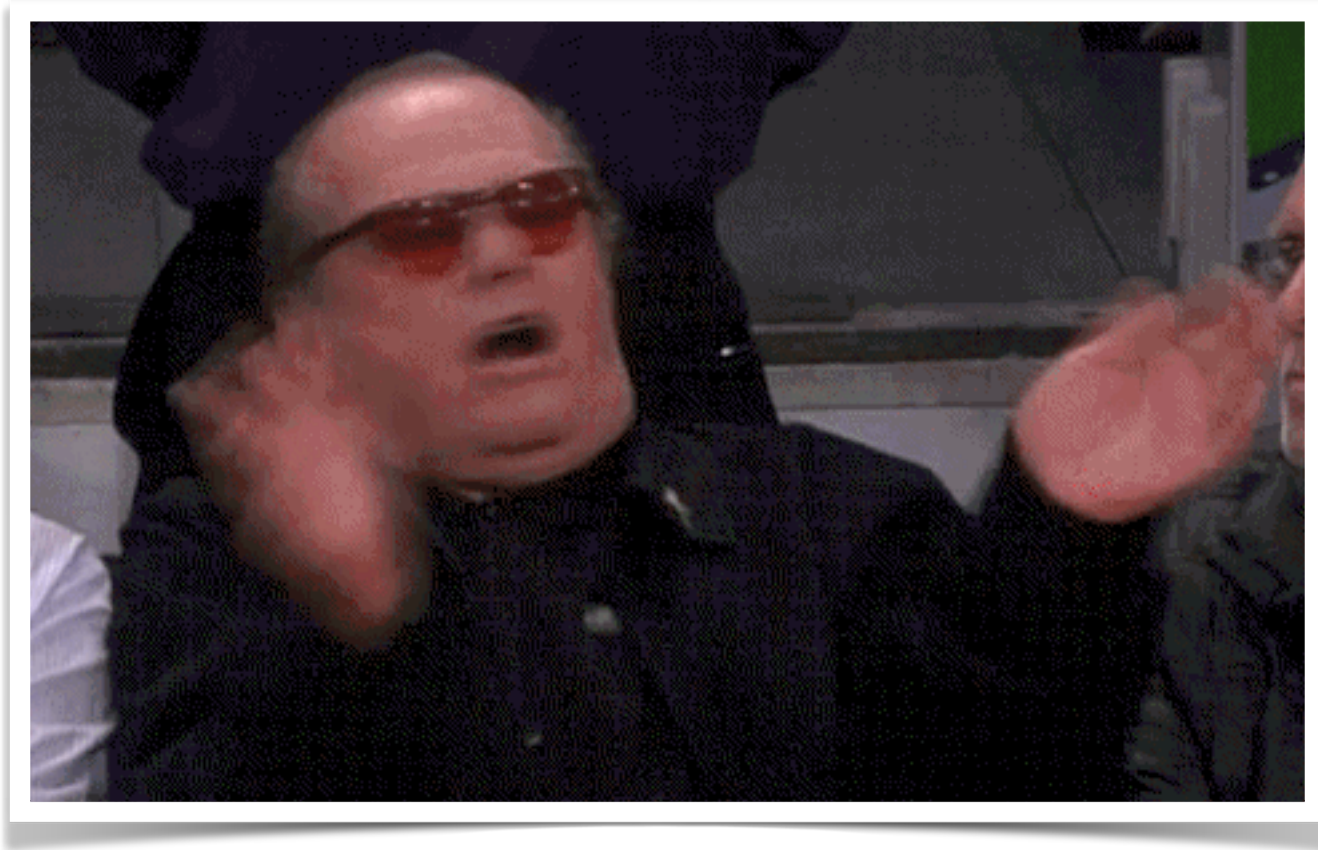
linus@example.com

margaret@example.com

tim@example.com

11

Mais c'est quoi ce truc ?

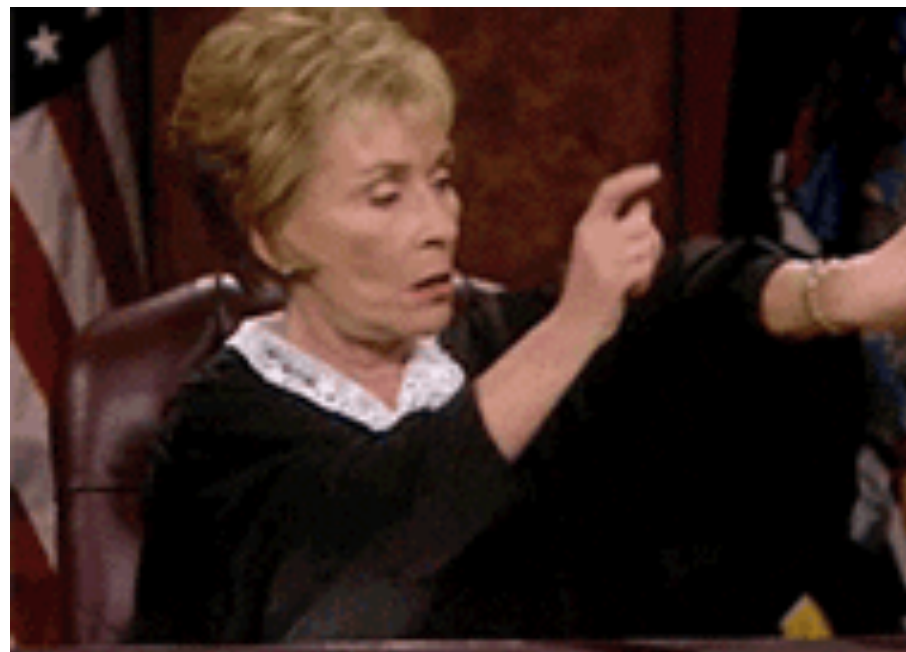
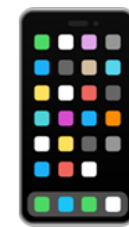


Mais c'est quoi ce truc ?

- Lecture ou mise à jour ?
 - 700 lectures pour 1 mise à jour
- Tant pis pour les mises à jour... mais :
 - Requêtes beaucoup plus simples
 - Moins de temps de latence



Et donc avec Flutter



Step by step

Activer les modules sur Firebase

The screenshot shows the Firebase console interface. On the left sidebar, the 'Firestore Database' module is highlighted with a red arrow. The main area displays the 'Cloud Firestore' interface, specifically the 'Données' (Data) tab for a collection named 'teams'. A red arrow points from the 'Firestore Database' module in the sidebar to the 'Données' tab in the main area.

Left Sidebar:

- Vue d'ensemble du p...
- Raccourcis de projet
- Authentication
- Firestore Database**
- Storage
- Catégories de produits
- Créer
- Publier et surveiller
- Analytics
- Engager
- Tous les produits

Main Area:

Tictic 2023

Cloud Firestore

Données Règles Index Utilisation Extensions

Protégez vos ressources Cloud Firestore des utilisations abusives telles que la fraude à la facturation et le hameçonnage [Configurer App Check](#)

[Vue Panneau](#) Générateur de requêtes

home > teams > 1ZPAir3V3ZxdE..

(default) teams 1ZPAir3V3ZxdEyTMY0WE

+ Commencer une collection + Ajouter un document + Commencer une collection

teams users 1ZPAir3V3ZxdEyTMY0WE

RHta1A1kGBRfNye1fRd7
aSnY02YqMrzAr51bFRHo
up9NwPKroIkNUINKXb07

+ Ajouter un champ

picturePath: "assets/img/team-2.png"

startDate: 15 septembre 2023 à 00:00:00 UTC+2

tags

- 0 "Web Development"
- 1 "Design"
- 2 "Networking"

title: "Web Weavers"

transactions

- 0 amount: 20

Step by step

Donner les droits nécessaires

Cloud Firestore

Données Règles Index Utilisation Extensions

i Vous cherchez le graphique de surveillance des règles ? Il se trouve désormais dans l'onglet [Utilisation](#) pour vous offrir une vue consolidée des métriques. Ignorer

Développer et tester

★ oct. 16, 2023 • 10:44 PM

○ oct. 16, 2023 • 10:44 PM

○ oct. 16, 2023 • 10:27 PM

```
1 rules_version = '2';
2
3 service cloud.firestore {
4   match /databases/{database}/documents {
5     match /{document=**} {
6       allow read, write: if true;
7     }
8   }
9 }
```

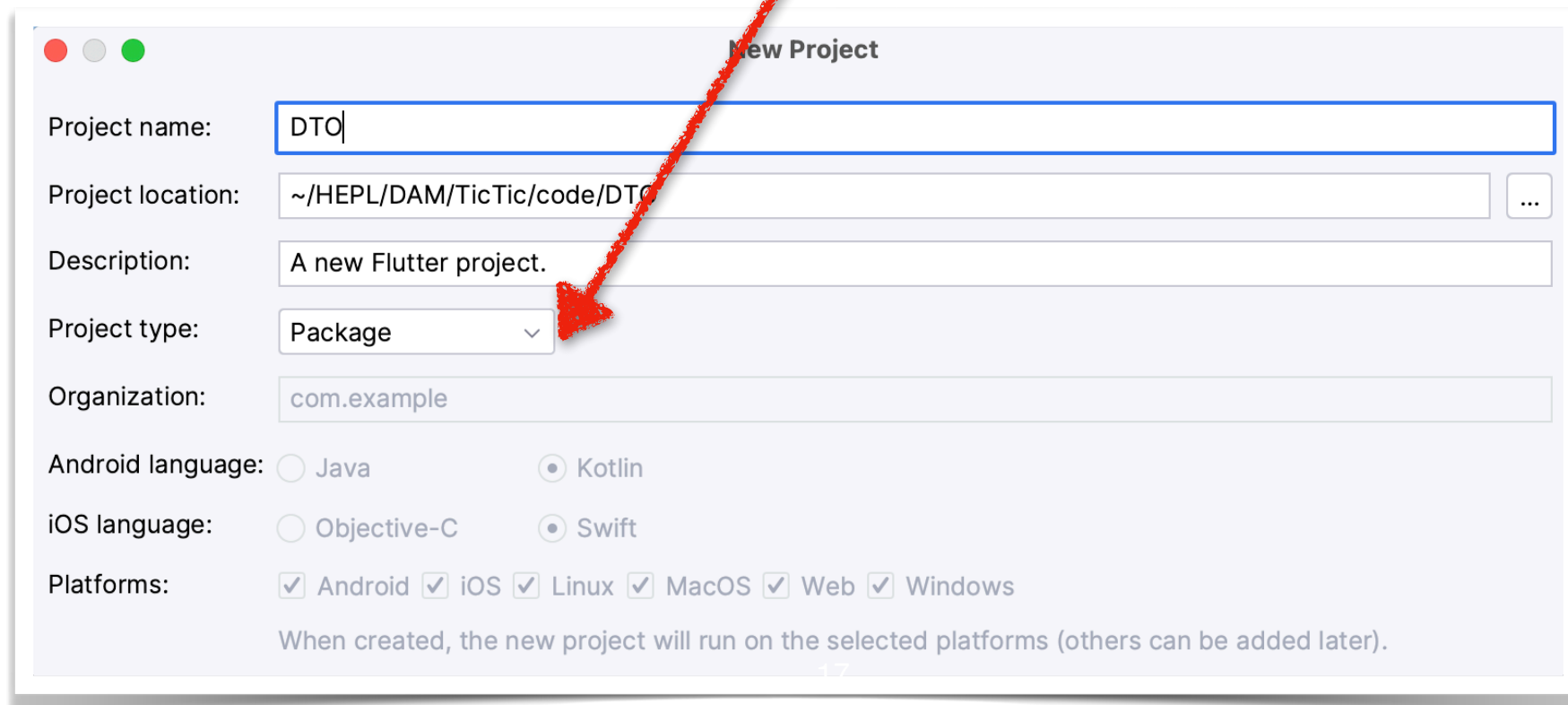
 Espace de test dédié aux règles
Experiment and explore with Security Rules



Step by step

Écrire des classes DTO (Objet de transfert de données)

- Objet de transfert de données
- Des classes très simples qu'on peut **sérialiser** et **désérialiser**
- Dans un projet Flutter à package Flutter



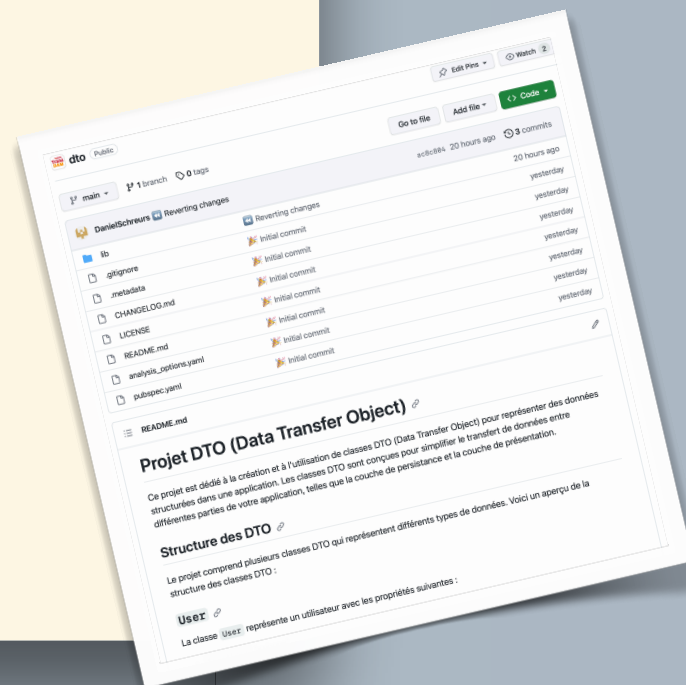
Pour récupérer les données

Écrire des classes DTO

```
class User {
  final String firstName; final String lastName; final String email;
  const User({
    required this.firstName, required this.lastName, required this.email,
  });

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      firstName: json['firstName'],
      lastName: json['lastName'],
      email: json['email'],
    );
  }

  Map<String, Object?> toJson() {
    return {
      'firstName': firstName,
      'lastName': lastName,
      'email': email,
    };
  }
}
```



Pour récupérer les données

Ajouter la référence

```
dependencies:  
  flutter:  
    sdk: flutter  
  cupertino_icons: ^1.0.2  
  flutter_svg: ^2.0.7  
  email_validator: ^2.1.17  
  intl: ^0.18.1  
  flutter_native_splash: ^2.3.2  
  firebase_auth: ^4.10.1  
  firebase_core: ^2.17.0  
  cloud_firestore: ^4.9.3  
  dto:  
    path: ../dto
```

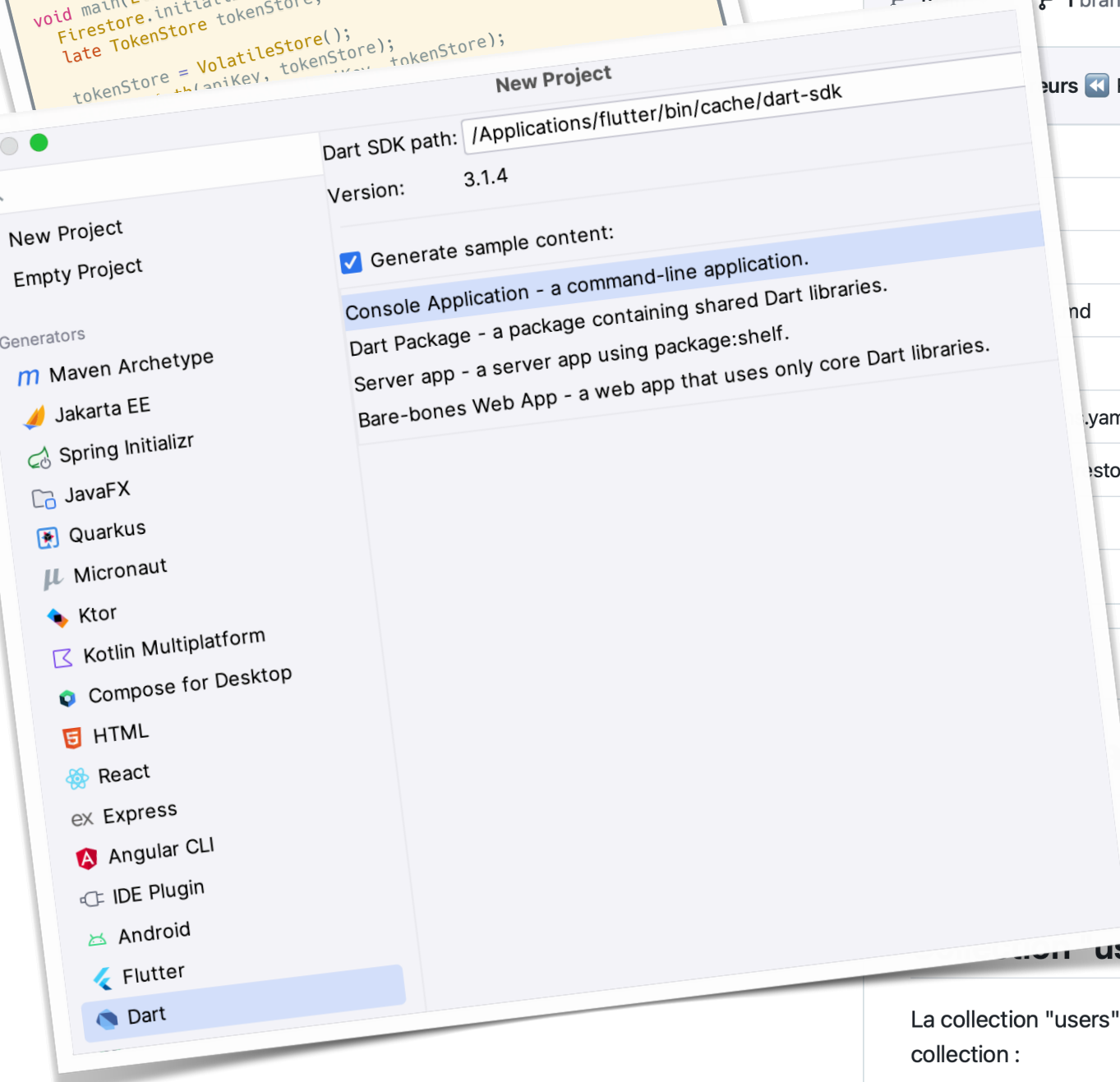
```
import 'package:dto/user.dart' as dto_user;  
  
@immutable  
class HomeScreen extends StatefulWidget {  
    
  class _HomeScreenState extends State<HomeScreen> {  
    late final Stream<DocumentSnapshot<dto_user.User>>? _userStream;  
  
    @override  
    void initState() {}  
  }  
}
```

Pour récupérer les données

(Un petit script pour générer des données 🧘)

```
String pi = PI;
String apiKey = API_KEY;

void main(List<String> arguments) async {
  Firestore.initialize(pi);
  late TokenStore tokenStore;
  tokenStore = VolatileStore();
  tokenStore = VolatileStore(tokenStore);
}
```



load_data_to_firestore Public

Edit Pins

Watch 2

main

1 branch 0 tags

Go to file

Add file

Code

Reverting changes

969a865 20 hours ago 4 commits

Initial commit	yesterday
Reverting changes	20 hours ago
Initial commit	yesterday
Initial commit	yesterday
Initial commit	yesterday
Initial commit	yesterday
Initial commit	yesterday
Initial commit	yesterday
Initial commit	yesterday
Initial commit	yesterday

to Firestore

atif de publier deux collections de données fictives dans une base de données Firestore. Les deux vantes :

users"

La collection "users" représente une série d'utilisateurs fictifs. Voici un exemple de données fictives pour cette collection :

```
import 'package:dto/user.dart';
```



Pour récupérer les données

Initialiser Firebase



```
Future<void> main() async {  
  initializeDateFormatting();  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  await FirebaseAuth.instance.setLanguageCode("fr");  
  runApp(const MyApp());  
}
```

Pour récupérer les données

Déclarer un flux de données



```
late final Stream<DocumentSnapshot<dto_user.User>>? _userStream;  
late final Stream<QuerySnapshot>? _teamsStream;
```

Un flux de données



Pour récupérer les données

Instancier le flux de données (team)

```
@override
void initState() {
  super.initState();
  if (FirebaseAuth.instance.currentUser != null) {
    _teamsStream = FirebaseFirestore.instance
      .collection('teams')
      .where('users',
        arrayContains: FirebaseAuth.instance.currentUser!.email)
      .withConverter<Team>(
        fromFirestore: (snapshot, _) => Team.fromJson(snapshot.data()!),
        toFirestore: (team, _) => team.toJson(),
      )
      .snapshots();
  }
}
```

users		(tableau) +	
0	"linus@example.com"		
1	"margaret@example.com"		
2	"tim@example.com"		

Pour récupérer les données

Instancier le flux de données (user)

```
@override
void initState() {
  super.initState();
  if (FirebaseAuth.instance.currentUser != null) {
    _userStream = FirebaseFirestore.instance
      .collection('users')
      .withConverter<dto_user.User>(
        fromFirestore: (snapshot, _) =>
          dto_user.User.fromJson(snapshot.data()!),
        toFirestore: (user, _) => user.toJson(),
      )
      .doc(FirebaseAuth.instance.currentUser!.email)
      .snapshots();
  }
}
```

+ Commencer une collection	+ Ajouter un document
teams	ada@example.com >
users >	alan@example.com
	⋮ grace@example.com
	linus@example.com
	margaret@example.com

Pour récupérer les données

Utiliser ce flux avec un `StreamBuilder<T>`



```
StreamBuilder<DocumentSnapshot<dto_user.User>>(
  stream: _userStream,
  builder: (BuildContext context, AsyncSnapshot<DocumentSnapshot<dto_user.User>> snapshot) {
    if (snapshot.hasError ||
        snapshot.connectionState ==
            ConnectionState.waiting) {
      return const HomeTitle(text: 'Bienvenue !');
    }
    return HomeTitle(
      text:
        'Hey, ${snapshot.data?.data()!.firstName}!');
  },)
```

Pour récupérer les données

Utiliser ce flux



```
StreamBuilder<QuerySnapshot>(
  stream: _teamsStream,
  builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(
        child: CircularProgressIndicator());
    }
    if (snapshot.hasError) {
      return const Text('Oups, une erreur est survenue !');
    }
    return Column(
      children: snapshot.data!.docs
        .map((DocumentSnapshot document) {
          Team team = document.data()! as Team;
          return TeamOverview(team: team);
        }).toList(),
    );
  },
),
```

Pour récupérer les données

Déclarer une fonction qui retourne un type Future<>

```
final FirebaseAuth _auth = FirebaseAuth.instance;
final FirebaseFirestore _firestore = FirebaseFirestore.instance;

Future<String> getUsername() async {
  final user = _auth.currentUser;
  final uid = user?.email;
  if (uid != null) {
    final userData = await _firestore.collection('users').doc(uid).get();
    return userData.data()?['firstName'] ?? '';
  }
  return '';
}
```

Future<String>

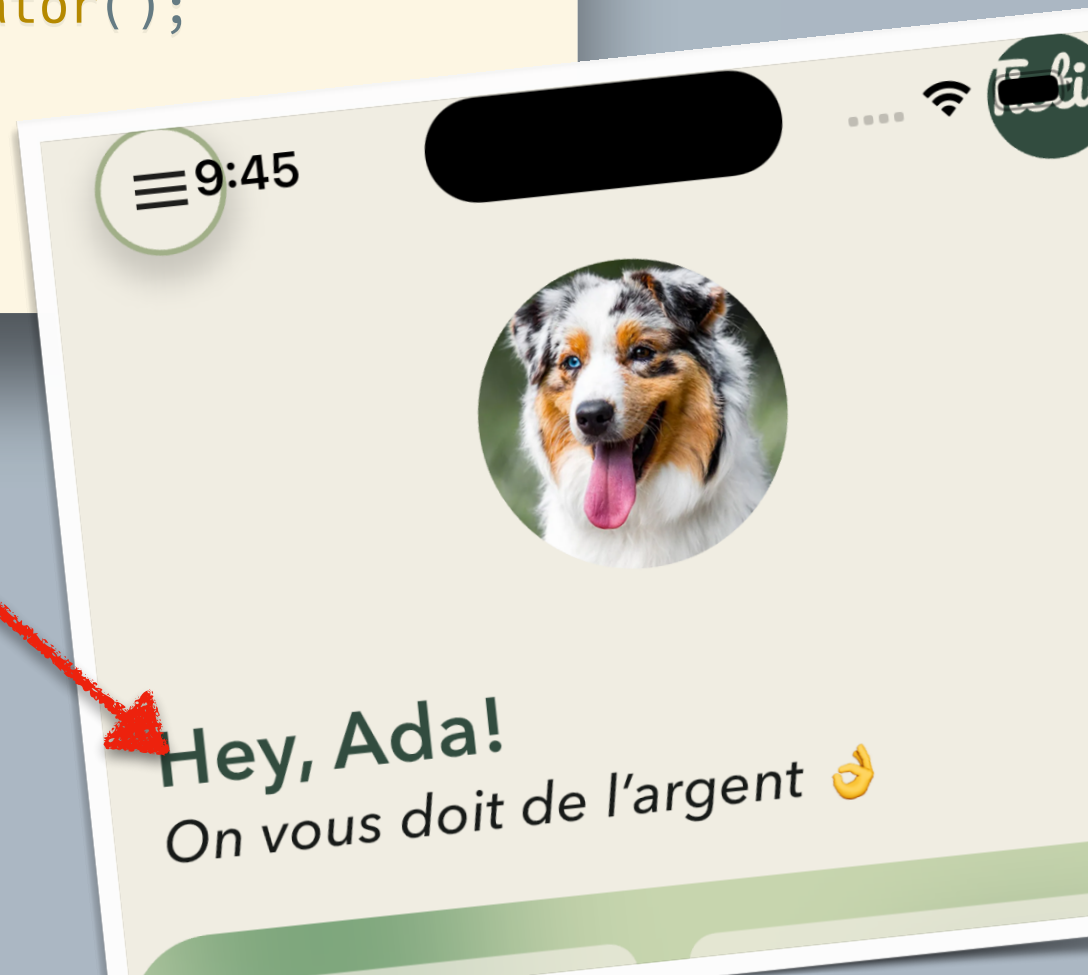


Pour récupérer les données

Déclarer une fonction qui retourne un type Future<>

```
FutureBuilder<String>(
  future: getUsername(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.done) {
      return Text('Hey, ${snapshot.data}');
    } else {
      return const CircularProgressIndicator();
    }
  },
),
```

```
Future<String> getUsername() async {}
```



Sources

Sources et ressources à consulter

- [Get to know Cloud Firestore](#)
- [The Firebase Database For SQL Developers](#)
- [NoSQL data modeling](#)
- firebase.google.com
- [Firebase : Podcast](#)